Toolkit Decoupling: Declarations → Backend

MICS Backend — Conceptual Architecture Plan

Context

Currently, MICS task plugin classes hardcode their identity in Python (HARDWARE, PARAMS, FLAGS, SEMANTIC_HARDWARE dicts). This locks hardware config to a specific Pi. The goal is to move all declarative config to the backend/UI, leaving plugin files as optional behavior-only extensions.

Core Idea

The Pi becomes a dumb executor. Everything that describes *what* a task needs comes from the backend at run time. The Pi only provides *how* to execute (hardware drivers + optional custom state functions).

What Moves to the Backend

Currently on Pi	Moves to
HARDWARE dict	Per-pilot hw config stored in DB, sent in START payload
PARAMS dict	Toolkit definition in DB (already partially there)
FLAGS dict	Toolkit definition in DB
SEMANTIC_HARDWARE dict	Toolkit definition in DB
prefs.json hardware section	Physical mapping (pins, polarity, pulse width) — editable per-pilot in UI, sourced from HANDSHAKE; Pi prefs.json remains as local fallback
FDA / state machine	Already in DB as fda_json

What Stays on the Pi

- Hardware driver modules (gpio.Solenoid_mics, i2c.Touch_Detector, etc.)
- mics_task base class with init_hardware() / init_flags() that accept incoming config
- Optional plugin file: only if the toolkit needs custom state functions beyond what mics_task provides

Plugin File: Optional

- If a toolkit needs no custom states → task_type = "mics_task", no plugin file needed, intentional
- If custom states exist → slim plugin class inherits mics_task, contains only state functions, no declarations

- Plugin file stored as blob in the toolkit DB record

Missing Plugin File: Auto-provision on Start

HANDSHAKE stores the pilot's available plugin class names in the DB. The API checks this at start-run time (before the orchestrator is involved):

```
User hits START
  → API checks DB: does this pilot's stored class list include "LearningCage"?
  → If missing: API instructs orchestrator to send INSTALL_PLUGIN first
  → Orchestrator sends file content (from toolkit DB record), waits for ACK
  → Then sends START as normal
  → No user intervention, no restart
```

Pi handles INSTALL_PLUGIN by writing file to PLUGINDIR and importing directly via importlib (bypasses _IMPORTED lock — no process restart needed).

Implementation Phases

Phase 1 — Pi Infrastructure

- mics_task.__init__ reads hw_config, semantic_hw, flags_def, params from START kwargs
- init_hardware() accepts hw_config from kwargs (physical mapping: pins, pulse width, polarity); falls back to local prefs.json only if not provided
- init_flags() accepts flags_def solely from kwargs — flags have no Pi-side definition, always from backend
- Strip declarations from existing plugin classes (keep state methods only)
- pilot.py: if task_type class not found → trigger auto-provision flow

Phase 2 — Backend / Orchestrator

- hw_config, flags_def, semantic_hw, params defined in UI and stored in existing toolkit DB tables
- START payload builder includes full bundle: hw_config + semantic_hw + flags_def + FDA
- hw_config sourced from pilot's prefs DB field (physical layer from HANDSHAKE) as base
- Pre-run validation: pilot's callable_methods covers all FDA-referenced state functions
- task_toolkits DB record gains plugin_file_content field

Phase 3 — UI

- Per-pilot hardware config editor (mirrors prefs.json HARDWARE section)
- Toolkit definition page: semantic_hw, flags, params (already partially exists)
- Validation: declared hw_config references valid entries in pilot's prefs.json

Phase 4 — Plugin Migration (future)

- If Pi is missing a required plugin class, orchestrator pushes file content before starting run
- Pi writes to PLUGINDIR, triggers re-import via importlib

Benefits

Multi-device portability — The same task definition runs on any Pi. Hardware wiring differences are configuration, not code.

No Pi code changes to add/modify tasks — New toolkits are defined entirely in the UI. Plugin files are only needed for genuinely unique behavior.

No version drift — HARDWARE/PARAMS/FLAGS no longer live in Python files that diverge across machines. One source of truth in the DB.

Auto-provisioning — A Pi that's never seen a toolkit gets its plugin file automatically on first run. No manual file copying.

Graceful fallback — If a toolkit has no custom states, it runs on mics_task directly with zero plugin overhead.

Validated before run — Orchestrator confirms hw_config matches prefs.json and callable_methods covers all FDA state references before the task starts.

Future state-builder compatible — This architecture is the natural foundation for a UI state builder where states are assembled from agreed primitive functions with no Python required at all.

Verification

• Start a task with task_type = "mics_task" (no plugin file) with hw_config pushed from backend — hardware initializes correctly

• Start a task using a slimmed plugin class — custom state functions available, declarations gone

• Start same task definition on two different Pis with different hw_config — each uses its own wiring